36

Agentic AI Concepts

AI that reasons, plans, and acts on its own

From answering questions to completing tasks

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	AI that can take actions to reach a goal
vs a chatbot	Acts, not just talks
The loop	Think → Act → Observe, repeated
Powered by	An LLM brain + tools + memory
Your project	The AI Agent project on your roadmap

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	**What is an AI Agent?**	From talking to doing
2	**Agent vs Plain LLM**	The key difference
3	**The Agent Loop**	Think, act, observe
4	**The Four Parts**	Brain, tools, memory, planning
5	**Tools — The Hands**	How agents act on the world
6	**Planning**	Breaking a goal into steps
7	**A Worked Example**	An agent in action
8	**ReAct — A Common Pattern**	Reason + Act together
9	**Strengths and Risks**	What to watch for
10	**Common Mistakes**	What to avoid
★	**Cheatsheet**	Quick reference

1. What is an AI Agent?

So far, LLMs have answered questions and written text. An AI agent goes further: it can take ACTIONS to complete a task. Give it a goal, and it figures out the steps, uses tools to do them, checks its progress, and keeps going until the job is done. It doesn't just talk — it acts.

The shift is from 'answer my question' to 'achieve my goal'. Instead of 'what's the weather in Paris?', you can say 'find the weather in every city I'm visiting next week and put it in a table'. An agent breaks that down, looks up each city, gathers the weather, and builds the table — all by itself.

> ■ *This is your next project frontier: the AI Agent project on your roadmap, where you'll wire your RAG chatbot in as a tool the agent can use. This topic builds the mental model before the hands-on frameworks (PydanticAI, CrewAI) coming next.*

2. Agent vs Plain LLM

The difference is action. A plain LLM takes input and produces text — one shot, then done. An agent runs in a loop: it thinks, does something, sees what happened, and decides the next move, repeating until the goal is reached.

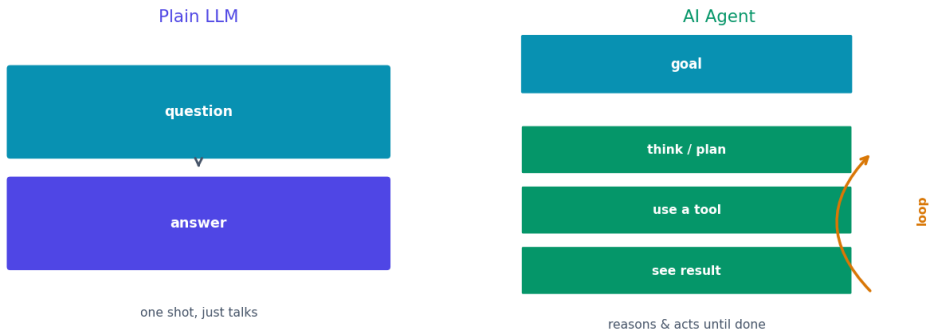

Fig 1 — A plain LLM answers once. An agent loops: thinking, acting, and observing until the goal is met.

	Plain LLM	AI Agent
What it does	Generates text	Takes actions to reach a goal
Steps	One response	Many, in a loop
Tools	None	Uses search, APIs, code, etc.
Decisions	You decide everything	It decides its own next steps
Good for	Answering, writing	Completing multi-step tasks

3. The Agent Loop

At the heart of every agent is a simple loop with three steps, repeated until the task is done. This is the single most important idea in agentic AI.

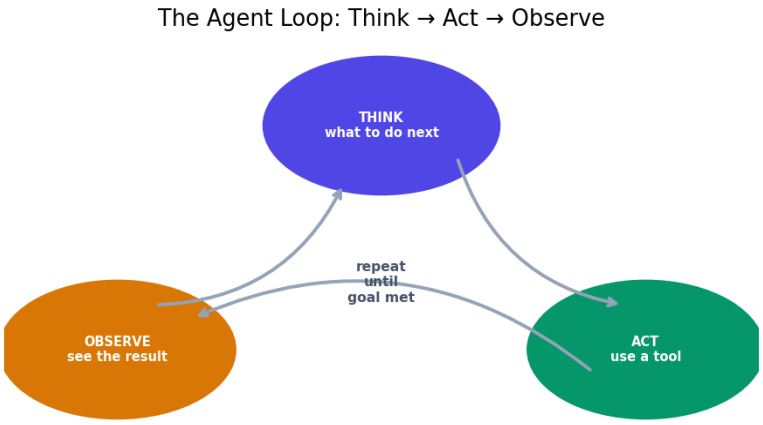

Fig 2 — The agent loop: Think (plan the next step), Act (use a tool), Observe (see the result), repeat.

Step	What happens
Think	The LLM reasons about what to do next to reach the goal
Act	It uses a tool — search, calculate, call an API, etc.
Observe	It looks at the tool's result
Repeat	Using what it observed, it thinks again and continues

The loop continues until the agent decides the goal is achieved, then it gives the final answer. This think-act-observe cycle is what lets an agent handle tasks that need several steps and adjust along the way if something doesn't work.

4. The Four Parts of an Agent

An agent is built from four parts working together. You've met all of them in earlier topics — now they combine.

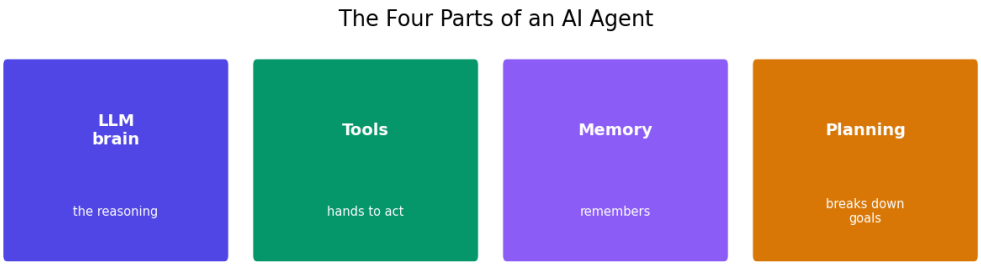

Fig 3 — The four parts: the LLM brain, tools to act, memory to remember, and planning to break down goals.

Part	Its role
LLM (brain)	Does the reasoning and decides what to do
Tools (hands)	Let it act: search, code, APIs, your RAG retriever
Memory	Remembers earlier steps and results
Planning	Breaks a big goal into smaller steps

The LLM is the brain that thinks. Tools are the hands that act. Memory holds what's happened so far. Planning breaks the goal into manageable pieces. Together, they turn a text-predictor into something that can get things done.

5. Tools — The Hands

Tools are what make an agent powerful. You learned about them in the LangChain topic: a tool is a function the agent can call to do something in the real world. Without tools, an agent can only think; with tools, it can act.

```
# A tool is just a function the agent can call
def search_web(query):
    '''Search the web for current information.'''
    return web_search_api(query)

def calculator(expression):
    '''Do exact math.'''
    return eval(expression)

def search_my_pdf(question):
    '''Search the user's documents (your RAG chatbot!).'''
    return rag_retrieve(question)

# Give these tools to the agent; it chooses when to use them
agent = create_agent(llm, tools=[search_web, calculator, search_my_pdf])
```

■ *See search_my_pdf? That's your RAG chatbot wired in as a tool — exactly your planned AI Agent project. The agent decides when a question needs your documents and calls it. Your existing project becomes one capability of a bigger agent.*

6. Planning

For complex goals, an agent first makes a plan — it breaks the big task into smaller steps before acting. This is like how you'd tackle a project: figure out the steps, then do them one by one.

Good planning is what separates a capable agent from one that flails. For 'book me a trip to Tokyo', a planning agent might decide: (1) find flights, (2) find hotels, (3) check the weather, (4) build an itinerary. Then it works through each step, using tools, adjusting if something fails. The LLM's reasoning ability is what makes this planning possible.

■ *Planning quality depends heavily on the LLM and the prompt. A clear goal and good instructions (your prompt engineering skills!) help the agent plan well. Vague goals lead to confused agents.*

7. A Worked Example

Let's watch an agent handle: 'Email me a summary of today's AI news.' A plain LLM can't do this — it can't browse or send email. An agent can, by chaining tools.

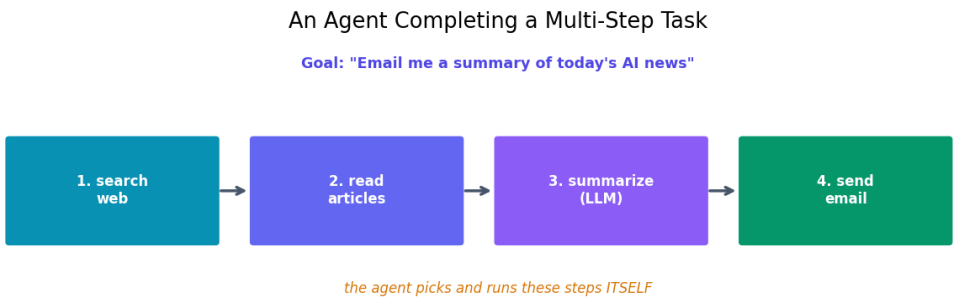

Fig 4 — An agent completing a multi-step task by choosing and running tools on its own.

The agent's loop in action: **Think** 'I need current news, so I'll search' → **Act** use the web search tool → **Observe** read the results → **Think** 'now summarise these' → **Act** use the LLM to summarise → **Think** 'now email it' → **Act** use the email tool → **Observe** sent → done. It picked and ran each step itself.

8. ReAct — A Common Pattern

ReAct (short for Reason + Act) is the most common way to build agents. It simply means the agent alternates between reasoning out loud and taking actions. At each step it writes its thought, then takes an action, then sees the result — which is exactly the think-act-observe loop.

```
# A ReAct agent's thinking looks like this:

Thought: I need the population of Tokyo.
Action: search_web('population of Tokyo')
Observation: About 14 million.

Thought: Now I need it as a percentage of Japan.
Action: calculator('14 / 125 * 100')
Observation: 11.2%

Thought: I have the answer.
Final Answer: Tokyo holds about 11% of Japan's population.
```

■ *Writing out the 'Thought' before each action is chain-of-thought (Topic 30) applied to agents. It makes the agent reason more carefully and makes its behaviour easy to follow and debug. Most agent frameworks use this pattern.*

9. Strengths and Risks

Strengths	Risks to watch
Handles multi-step tasks	Can loop forever if confused
Uses real tools and data	May use a tool wrongly
Adjusts when things fail	Costs add up (many LLM calls)
Automates whole workflows	Can take unintended actions
Combines all your skills	Harder to predict and debug

Agents are powerful but need guardrails. Because they act on their own, you should limit what tools they have, cap how many steps they can take (to avoid infinite loops), and be careful giving them tools that take real actions (like sending emails or spending money). Start with safe, read-only tools while learning.

> ■ *A key safety habit: set a maximum number of steps. Without a limit, a confused agent can loop forever, burning time and money on LLM calls. Always cap the loop.*

10. Common Mistakes

Mistake	Why it's a problem	Fix
No step limit	Agent loops forever	Cap the max number of steps
Too many tools	Agent gets confused	Start with a few clear tools
Vague goals	Bad planning, wrong actions	Give clear, specific goals
Risky tools too early	Unintended real actions	Use safe read-only tools first
Ignoring cost	Many LLM calls add up	Watch usage; limit steps
No tool descriptions	Agent misuses tools	Describe each tool clearly

Quick Reference Cheatsheet

Concept	Meaning
AI agent	an LLM that acts to reach a goal
vs plain LLM	acts and loops, not just answers
Agent loop	Think -> Act -> Observe, repeat
Think	reason about the next step
Act	use a tool
Observe	see the tool's result
Four parts	LLM brain, tools, memory, planning

Tool	a function the agent can call
Planning	break a goal into steps
ReAct	reason + act pattern (most common)
Key safety	cap the number of steps
Your project	RAG chatbot wired in as a tool

The one idea to remember: an AI agent is an LLM that doesn't just answer — it acts. Using a Think-Act-Observe loop, plus tools, memory, and planning, it works through multi-step tasks on its own until the goal is reached.

Next Topic: PydanticAI — a clean, modern framework for building reliable agents in Python, with a strong focus on structured, predictable outputs.